jmbayes: fast, memory-efficient, data-parallel Bayesian joint models for longitudinal and time-to-event data

Mo Assaf
Neuratom
assaf@neuratom.ai

Abstract

We present **jmbayes**, a data-parallel reimplementation of the R package JMbayes2, a widely used tool for Bayesian joint models of longitudinal and time-to-event data. It fits the same model, with the same priors and the same Markov chain Monte Carlo (MCMC) algorithm, and gives statistically equivalent results. On the largest problem we tested it was 10× faster with 27× less memory, and the gap grew with problem size. For 3,120 patients and 90,000 iterations, JMbayes2 took 104 minutes and 37.8 GB of RAM; jmbayes took 11 minutes and 1.4 GB. The speed-up grows with the problem: 3.3× on 312 patients, 8–9× on 1,560–3,120 patients at JMbayes2's default run length, and 9.7–10.5× on long runs with 3,120 patients, still growing at the largest size tested. jmbayes's memory stayed at 1.3–1.4 GB of RAM and under 0.2 GB of GPU memory in every run, up to 90,000 iterations, while JMbayes2's grew with patients × iterations beyond the capacity of a typical workstation. The gains come from restructuring the computation, not from changing the method: chains and patients are processed as data-parallel tensor operations on a GPU instead of one process per chain, each iteration is replayed as a recorded CUDA graph, and the fit statistics (DIC, WAIC, LPML) are computed exactly from running totals instead of a stored per-patient log-likelihood for every draw. In formal equivalence tests on the primary biliary cholangitis (PBC) data, all 50 model parameters agreed with JMbayes2 within 0.2 posterior standard deviations, and the two programs differed from each other less than two JMbayes2 runs with different random seeds. jmbayes is open source (GPL-3.0) and installable with `pip install jmbayes`.

1 Introduction

Joint models link repeated measurements of a biomarker to the risk of an event, such as death or transplantation, through shared random effects [3, 2]. They are the standard way to study how a marker's trajectory relates to risk, and they give dynamic predictions that update as new measurements arrive. The R package JMbayes2 [1] fits a broad class of these models in a Bayesian framework, with an MCMC sampler written in C++.

Two practical limits appear as studies grow. First, run time: each MCMC iteration updates every patient's random effects, and models with several markers often need tens of thousands of iterations to converge. Second, memory: JMbayes2 stores the log-likelihood of every patient at every iteration to compute fit statistics after sampling, so its memory grows with patients × iterations. In practice, long runs on cohorts of a few thousand patients no longer fit in the memory of a typical 16 GB workstation.

jmbayes removes both limits without changing the statistical method. It is a port of JMbayes2's sampler, step by step, to PyTorch [4], restructured so that the work of each iteration runs as data-parallel operations. On the same machine, it was 10× faster than JMbayes2 on the largest problems we tested, with a gap that grew with cohort size and run length, and its memory barely grew with run length. We show with formal equivalence tests that it gives the same results as JMbayes2 within Monte Carlo error.

2 Model

For patient i and marker k, the longitudinal sub-model is a generalised linear mixed model,

$$g_k\{E(y_{ik}(t) \mid b_{ik})\} = \eta_{ik}(t) = x_{ik}(t)^\top \beta_k + z_{ik}(t)^\top b_{ik}, \qquad b_i = (b_{i1}, \ldots, b_{iK}) \sim N(0, D),$$

with an identity link for continuous markers and a logit link for binary ones. The hazard of the event is

$$h_i(t) = h_0(t) \exp\Big\{\gamma^\top w_i + \sum_k \alpha_k \, f_k(\eta_{ik}, t)\Big\},$$

where w_i are baseline covariates and f_k is the functional form that links marker k to the hazard: its current value $\eta_{ik}(t)$, its slope, or its cumulative area. The log baseline hazard is a B-spline with a penalised (random-walk) prior whose precision is estimated. Priors, starting values and all MCMC updates follow JMbayes2: a Metropolis-within-Gibbs sampler with adaptive random-walk proposals for the random effects, fixed effects without a matching random effect, association parameters and spline coefficients, and Gibbs or Metropolis steps for the covariance matrix D and the residual variances. Hierarchical centring of the fixed effects is applied as in JMbayes2.

3 Methodology

Where the parallelism is. We began by classifying the work in one MCMC iteration by the kind of parallelism it allows. The parameter blocks (random effects, covariance matrix, fixed effects, residual variances, baseline hazard, survival and association coefficients) must be updated in order, because each update conditions on the latest values of the others, and iterations must run one after another, because the chain is Markov. That part is sequential. Within each block, however, the same arithmetic is applied to many independent units: every patient's random effects are updated with the same proposal and acceptance rule, every observation's linear predictor and likelihood term has the same form, and the survival integral is evaluated at the same quadrature points for every patient. This is data parallelism: one instruction stream applied to many data elements (SIMD, or SIMT on a GPU). The chains add a second level: they are fully independent, so they are task parallelism (MIMD).

JMbayes2 exploits only the MIMD level: it runs each chain in its own R process on its own CPU core, and each process loops over patients in compiled C++. With the default 3 chains, 3 cores are busy whatever the cohort size, and each process keeps its own copy of the data and of the stored draws. jmbayes recasts the chains as data parallelism as well: it adds the chain as a leading tensor dimension, so a single instruction stream updates all patients of all chains at once. On a GPU, with thousands of parallel lanes, an iteration then costs almost the same for 312 or 3,120 patients until the device is saturated, which is consistent with the 1.4-fold increase in MCMC time we measured for ten times the patients (Section 6). On a CPU, the same code uses vector instructions and threads but far fewer lanes, which is why the CPU version is not faster than JMbayes2. Because iterations themselves cannot be parallelised, the fixed cost of launching many small GPU operations in every iteration matters, which led to the use of CUDA graphs (Section 4).

Reuse instead of new compiled code. JMbayes2 needs its sampler in C++ because loops in R are slow. We asked instead which parts of JMbayes2 already have an equivalent in the Python scientific stack, and wrote only what was missing. Data handling uses pandas; B-spline bases come from SciPy; the linear mixed models that provide starting values and centre some priors (R's `lme`) are fitted with the REML estimator of statsmodels; starting values for the logistic model come from a statsmodels GLM; numerical optimisation uses SciPy and Gauss–Hermite nodes come from NumPy. Two estimators had no Python version with exactly R's behaviour and were written directly: a binomial mixed model by maximum likelihood with

adaptive Gauss–Hermite quadrature (as in R's `GLMMadaptive`) and a Cox model with Efron's handling of ties (as in R's `coxph`). The MCMC sampler itself is written with PyTorch tensor operations.

This removes the need for any compiled code of our own. Python only orchestrates: every numerical operation runs in the compiled back-ends of these libraries, in C++ and CUDA for PyTorch and in C and Fortran (BLAS and LAPACK) for NumPy and SciPy. On the GPU, each MCMC iteration is a single replay of a recorded CUDA graph, so the Python interpreter is not in the inner loop at all. The package is therefore pure Python, distributed as one platform-independent wheel that runs wherever PyTorch runs.

Porting and verification. The sampler was ported from JMbayes2's C++ source function by function, keeping the same update order, proposals, adaptation rules and priors, so that the only intended difference is the random-number stream. Much of this translation was done with the assistance of Claude, a large language model from Anthropic, which also identified the SIMD and MIMD parallelism opportunities described above. The author reviewed the code, and correctness was established empirically rather than assumed from the translation: unit tests compare the design matrices, starting fits and priors with JMbayes2's output, and the full model was tested for statistical equivalence with JMbayes2 (Section 5). Where the port revealed inconsistencies in JMbayes2's own code, we reproduced them in the sampler so that results stay comparable, and documented them.

4 Implementation

Batched chains and patients. Every quantity is stored with a leading chain dimension, and all patients' random effects of all chains are updated in one tensor operation. Marker trajectories are updated incrementally when random effects change instead of being rebuilt.

CUDA graphs. An MCMC iteration launches many small GPU kernels. jmbayes records one full iteration as a CUDA graph the first time it runs and replays the recorded graph afterwards, which avoids most of the per-kernel launch overhead. All state stays on the GPU; only the kept parameter draws are copied back.

Fit statistics without the log-likelihood table. Let $\ell_i^{(s)}$ be the log-likelihood of patient i (markers, event and random effects) at kept draw s, for $s = 1, \ldots, S$. JMbayes2 stores the $S \times n$ matrix of $\ell_i^{(s)}$ and computes, after sampling,

$$\begin{aligned}
\mathrm{DIC} &= \bar{D} + p_D, \quad \bar{D} = -\tfrac{2}{S}\textstyle\sum_s \sum_i \ell_i^{(s)}, \quad p_D = \bar{D} - D(\bar{\theta}), \\
\mathrm{WAIC} &= -2\textstyle\sum_i \log\left(\tfrac{1}{S}\sum_s e^{\ell_i^{(s)}}\right) + 2\sum_i \mathrm{Var}_s\left(\ell_i^{(s)}\right), \\
\mathrm{LPML} &= \textstyle\sum_i \log \mathrm{CPO}_i, \quad \mathrm{CPO}_i = \left(\tfrac{1}{S}\sum_s e^{-\ell_i^{(s)}}\right)^{-1}
\end{aligned}$$

[5, 6, 7]. Each of these needs only a few running totals per patient: $\sum_s \ell_i^{(s)}$; $\log \sum_s e^{\ell_i^{(s)}}$ and $\log \sum_s e^{-\ell_i^{(s)}}$, accumulated on the log scale with a stable log-add-exp (the exponentials themselves under- or overflow); and the variance of $\ell_i^{(s)}$, accumulated with Welford's algorithm [8] and combined across chains with the pairwise formula of Chan et al. [9]. jmbayes updates these totals at every kept draw and never stores the matrix, so memory depends on the number of patients but not on the run length. The totals are kept in 64-bit precision; the values agree with those computed from the full matrix to within 10^{-8} relative error. For analyses that need every draw, such as PSIS-LOO and its Pareto-k diagnostic [10], the matrix can still be kept with `fit(..., save_loglik=True)`.

Precision. jmbayes samples in 32-bit floating point by default and refits automatically in 64-bit if a run produces invalid values. Over 9 matched benchmark settings on the GPU, 32-bit was 1.67× faster than 64-bit (geometric mean; 95% CI 1.49–1.86), and its results were equivalent to JMbayes2 in the same tests as 64-bit (Section 5). We rejected bfloat16: it keeps only 8 significant bits, so near a log-likelihood of 1,000 neighbouring representable values are 4 apart, too coarse for MCMC acceptance decisions.

Deliberate differences. While porting we found three places where JMbayes2's C++ code appears inconsistent. Two affect the sampler (a random-effects density not refreshed after the fixed-effects update, and a survival denominator not refreshed after the smoothing-parameter update); jmbayes reproduces both so that results match. The third affects dynamic prediction: when sampling a new patient's random effects, JMbayes2 scales the survival term differently from the code that then computes the risk. jmbayes uses the consistent version by default and reproduces JMbayes2's with `predict(..., r_sampling=True)`.

Interface. Models are written with Python objects (`Marker`, `Event`, `JointModel`) or with JMbayes2-style formula strings; both build the same model and give identical draws. jmbayes does not impute or silently drop data: missing or invalid values stop the fit with a message naming the columns.

5 Equivalence with JMbayes2

Setup. We used the PBC data [11] (312 patients, 1,945 visits, 169 deaths or transplants) and the model from the JMbayes2 documentation: a survival model with sex, linked through the current value of log serum bilirubin and prothrombin time (linear mixed models) and ascites (logistic mixed model), each with a random intercept and slope. Each program was run twice with different seeds, with 12 chains of 30,000 iterations (5,000 burn-in), giving 300,000 kept draws per run.

Error bars. To compare MCMC estimates we need their Monte Carlo standard errors (MCSE). Comparing two runs of the same program, the MCSE reported by JMbayes2 understated the real run-to-run error: only 80% (R) and 88% (Python) of differences fell within ±2 MCSE. We therefore inflated all MCSEs by the factor $k = 1.40$ estimated from these same-program pairs; the correction applies equally to both programs.

Test. For each of the 50 parameters, d is the difference between the jmbayes and JMbayes2 posterior means (each pooled over its two runs). The programs are declared equivalent for that parameter if the 90% confidence interval of d lies within $\pm\delta$ (two one-sided tests at $\alpha = 0.05$ [12]), with δ equal to 0.1 or 0.2 posterior standard deviations (SD).

Results. All 50 parameters were equivalent at ±0.2 SD and 43 at ±0.1 SD; none was shown to differ. The median absolute difference between jmbayes and JMbayes2 was 0.011 SD (largest 0.042), smaller than between two JMbayes2 runs (0.023 SD, largest 0.141) and two jmbayes runs (0.018 SD, largest 0.070). All 15 regression coefficients led to the same conclusions in both programs (direction, significance at 5%, and whether the 95% credible interval excludes zero). Table 1 and Figure 1 show the hazard ratios.

Fit statistics agreed within run-to-run variation (relative differences below 0.3%). Convergence was also the same: the Gelman–Rubin $\hat{R}$ [13] of every regression coefficient was below 1.05 in all four runs, the largest being 1.043 in both programs, for the same parameters (those of the ascites sub-model). With JMbayes2's default length (3 chains, 3,500 iterations), neither program converged for the ascites parameters, at any cohort size.

Table 1: Hazard ratios (posterior mean, 95% credible interval) from the same model and data. Markers act through their current value.

Effect on the hazard of death or transplantation	JMbayes2	jmbayes
Female sex	0.52 (0.26–1.03)	0.51 (0.25–1.03)
log serum bilirubin (per unit)	1.68 (1.18–2.33)	1.68 (1.18–2.31)
Prothrombin time (per second)	1.01 (0.79–1.25)	1.00 (0.78–1.25)
Ascites (per unit log-odds)	1.77 (1.40–2.39)	1.77 (1.40–2.40)

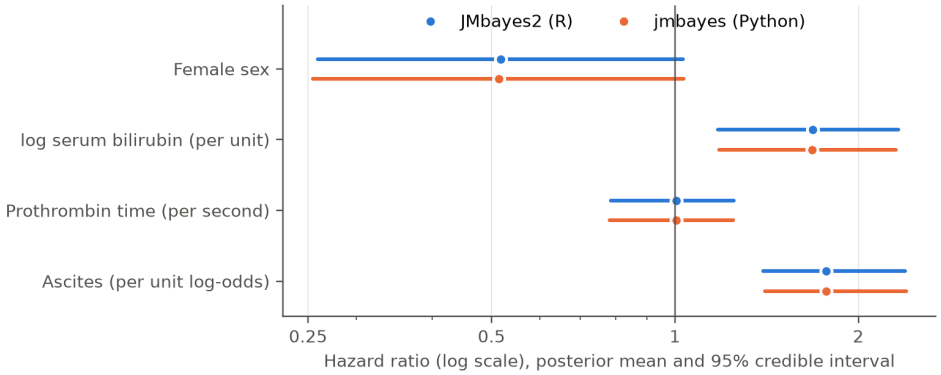


Figure 1: Hazard ratios with 95% credible intervals from JMbayes2 and jmbayes.

For dynamic predictions, we gave jmbayes JMbayes2's own posterior draws; with JMbayes2's random-effects sampling (`r_sampling=True`), jmbayes's predicted risks matched JMbayes2's to within 0.01. With its default, consistent sampling, risks can differ by a few percentage points; for PBC patient 93, the one-year risk was 0.63 instead of 0.70.

Competing risks. jmbayes also fits cause-specific hazards for competing events, the model JMbayes2 fits with `crisk_setup()` and `strata()`: each cause has its own baseline hazard, covariate effects and marker associations, and predictions give the cumulative incidence of each cause. We fitted death and transplantation in PBC as competing events, with cause-specific effects of sex and of the current values of log serum bilirubin and prothrombin time, in both programs (two seeds each, 12 chains of 30,000 iterations). None of the 50 parameters was shown to differ; 36 were shown equivalent within ±0.2 posterior SD and 27 within ±0.1 SD, the rest being inconclusive at this run length. The largest difference between the programs, 0.065 SD, was smaller than between two JMbayes2 runs with different seeds (0.18 SD). Hazard ratios agreed to two decimals; for example, per unit of log bilirubin, 3.27 (JMbayes2) and 3.27 (jmbayes) for death and 3.66 and 3.64 for transplantation. JMbayes2 0.6-5's `predict()` fails on this model, so cumulative incidences were checked internally: summed over causes, they matched one minus the overall survival to within 0.004.

Prediction accuracy. jmbayes also ports JMbayes2's measures of predictive accuracy for dynamic predictions: time-dependent ROC curves and AUC, the Brier score (also integrated over the prediction window), and calibration (ICI, E50, E90 and the calibration curve), with model-based or inverse-probability-of-censoring weights. Given the same predicted risks, they reproduced JMbayes2's values to 10^{-10} or better in two prediction windows.

Table 2: MCMC time and peak RAM. jmbayes on the GPU in 32-bit precision; JMbayes2 on the CPU. Dashes: not run.

		JMbayes2		jmbayes		
Patients	Iterations	Time	RAM	Time	RAM	Speed-up
312	3,500	55 s	1.28 GB	17 s	1.28 GB	3.3×
624	3,500	87 s	1.45 GB	17 s	1.28 GB	5.0×
1,560	3,500	157 s	1.95 GB	20 s	1.29 GB	8.0×
3,120	3,500	202 s	2.41 GB	24 s	1.31 GB	8.6×
1,560	7,000	237 s	2.34 GB	41 s	1.29 GB	5.8×
1,560	14,000	716 s	3.36 GB	82 s	1.30 GB	8.8×
1,560	28,000	1,046 s	6.25 GB	164 s	1.31 GB	6.4×
3,120	7,000	450 s	3.44 GB	48 s	1.31 GB	9.4×
3,120	14,000	521 s	6.15 GB	96 s	1.32 GB	5.4×
3,120	28,000	2,056 s	12.0 GB	196 s	1.33 GB	10.5×
3,120	60,000	5,893 s[a]	25.2 GB	429 s	1.39 GB	
1,560	90,000	–	–	529 s	1.36 GB	
3,120	90,000	6,266 s	37.8 GB	643 s	1.38 GB	9.7×

[a] This run overlapped with other work on the machine. JMbayes2's time per iteration was 98 ms, against 70–73 ms at 28,000 and 90,000 iterations, so this time is likely inflated and no speed-up is given.

6 Speed and memory

Setup. To vary the cohort size, the PBC data were copied 1, 2, 5 and 10 times with new patient identifiers (312 to 3,120 patients). We fitted the same model with 3 chains (burn-in 500, thinning 5) at 3,500 iterations (JMbayes2's default) and, for 1,560 and 3,120 patients, up to 90,000 iterations. Each run was a separate process, run one at a time. Peak RAM is the resident memory of the whole process tree, sampled every 0.2 s, which includes JMbayes2's worker processes (one per chain). MCMC time excludes the initial separate model fits, which both programs need. JMbayes2 runs its chains in parallel on the CPU; jmbayes ran on the GPU in 32-bit precision. All runs used one machine: Intel Core i9-12900K (x86-64; 16 cores, 8 performance and 8 efficiency, 24 threads), 64 GB RAM, NVIDIA GeForce RTX 3090 (24 GB), Windows 11, R 4.6.2 with JMbayes2 0.6-5, Python 3.11 with PyTorch 2.6 (CUDA 12.4).

Results. Table 2 and Figure 2 show the results. jmbayes was 3.3× faster on 312 patients and 8.0–8.6× faster on 1,560–3,120 patients at the default length, and 9.7–10.5× on long runs: at 90,000 iterations for 3,120 patients, JMbayes2 took 104 minutes and 37.8 GB of RAM, and jmbayes 11 minutes and 1.4 GB. Ten times more patients increased jmbayes's MCMC time 1.4-fold and JMbayes2's 3.7-fold. jmbayes's RAM stayed at 1.3–1.4 GB and its GPU memory below 0.2 GB in every run, up to 90,000 iterations, while JMbayes2's memory grew with both patients and iterations.

The speed comes from the GPU. Run on the CPU alone, jmbayes was about as fast as JMbayes2 on small cohorts and slower on large ones (445 s vs 202 s for 3,120 patients at 3,500 iterations), but used less than half the memory (0.6–0.7 GB).

7 Limitations

Each benchmark setting was run once, so single timings carry noise; for example, JMbayes2 took less time for 3,120 patients at 14,000 iterations than for 1,560, and its run at 60,000

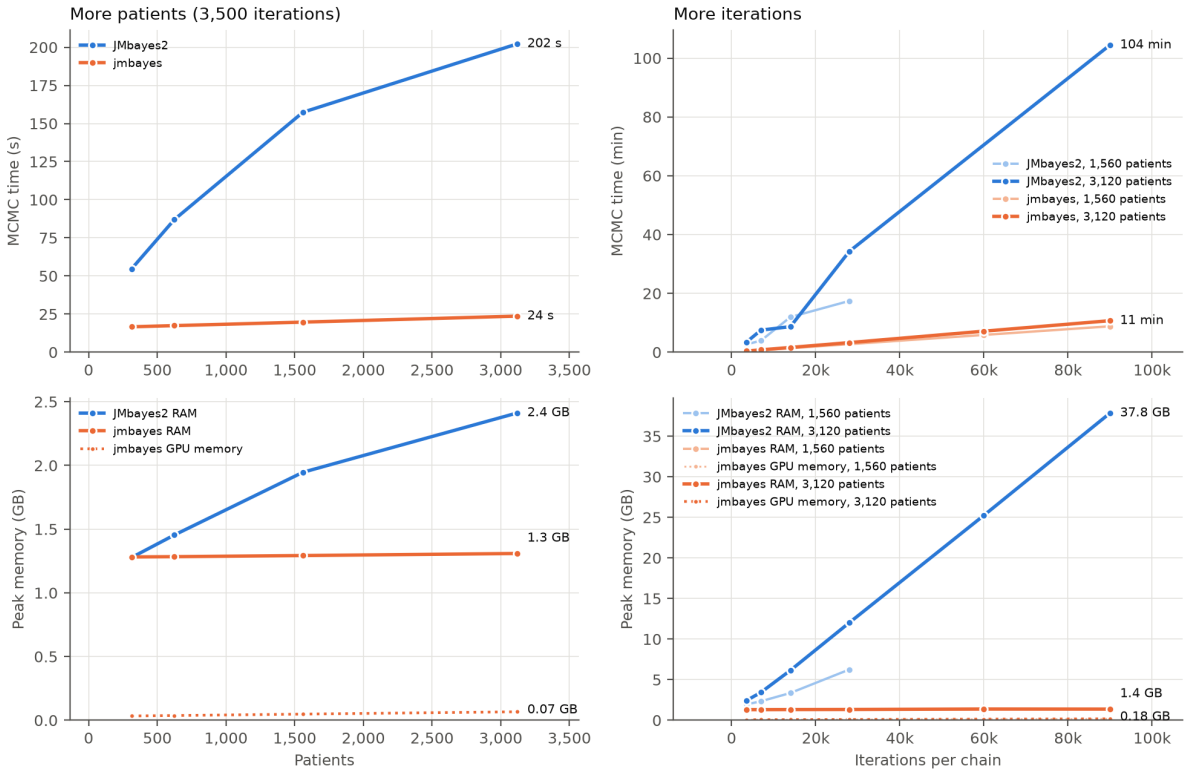


Figure 2: MCMC time (top) and peak memory (bottom) as the number of patients (left, 3,500 iterations) and the number of iterations (right; light lines 1,560 patients, dark lines 3,120) grow. Dotted: jmbayes GPU memory.

iterations overlapped with other work (Table 2). Larger cohorts were made by copying one data set, which tests scaling but not new data. Equivalence was tested in depth on one model and data set; three further models (splines, slope and area associations, fixed effects without random effects) agreed in smaller tests. The speed-up needs an NVIDIA GPU. jmbayes does not yet cover all of JMbayes2: it supports Gaussian and binomial markers, value, slope and area associations, right censoring and competing risks, but not other outcome families, strata other than competing risks, left truncation or interval censoring, and it does not compute marginal fit statistics. Unsupported models stop with an error rather than fitting a different model.

8 Availability

jmbayes 0.5.1 is available from PyPI (`pip install jmbayes`) and at `https://github.com/neuratom/jmbayes` under the GPL-3.0 licence, like JMbayes2. The repository contains the scripts and results for every number in this paper, including the full equivalence report. jmbayes is an independent reimplementation and is not affiliated with or endorsed by the authors of JMbayes2, whose work it builds on.

References

[1] D. Rizopoulos, G. Papageorgiou and P. Miranda Afonso. JMbayes2: Extended joint models for longitudinal and time-to-event data. R package, `https://github.com/drizopoulos/JMbayes2`.

[2] D. Rizopoulos. *Joint Models for Longitudinal and Time-to-Event Data: With Applications in R.* Chapman & Hall/CRC, 2012.

[3] A. A. Tsiatis and M. Davidian. Joint modeling of longitudinal and time-to-event data: an overview. *Statistica Sinica*, 14:809–834, 2004.

[4] A. Paszke et al. PyTorch: an imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, 2019.

[5] D. J. Spiegelhalter, N. G. Best, B. P. Carlin and A. van der Linde. Bayesian measures of model complexity and fit. *Journal of the Royal Statistical Society: Series B*, 64(4):583–639, 2002.

[6] S. Watanabe. Asymptotic equivalence of Bayes cross validation and widely applicable information criterion in singular learning theory. *Journal of Machine Learning Research*, 11:3571–3594, 2010.

[7] S. Geisser and W. F. Eddy. A predictive approach to model selection. *Journal of the American Statistical Association*, 74(365):153–160, 1979.

[8] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.

[9] T. F. Chan, G. H. Golub and R. J. LeVeque. Algorithms for computing the sample variance: analysis and recommendations. *The American Statistician*, 37(3):242–247, 1983.

[10] A. Vehtari, A. Gelman and J. Gabry. Practical Bayesian model evaluation using leave-one-out cross-validation and WAIC. *Statistics and Computing*, 27:1413–1432, 2017.

[11] P. A. Murtaugh, E. R. Dickson, G. M. Van Dam, M. Malinchoc, P. M. Grambsch, A. L. Langworthy and C. H. Gips. Primary biliary cirrhosis: prediction of short-term survival based on repeated patient visits. *Hepatology*, 20(1):126–134, 1994.

[12] D. J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.

[13] A. Gelman and D. B. Rubin. Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4):457–472, 1992.